GUIA TECNICA E INDUCCION

MedicalSys
Sprint 2 - MVP local con microservicios

Capacitacion practica para el primer dia: Docker, comunicacion entre servicios, contratos, datos, seguridad, flujo de trabajo y tareas por integrante.

Version 1.2 - 22 de agosto de 2026

Equipo MedicalSys - La Paz, Bolivia

Antes de comenzar

Esta guia asume que el equipo conoce Java y bases de datos a un nivel inicial, pero tiene poca experiencia con Docker y casi ninguna con microservicios. No pretende convertirlos en expertos en un dia. Su objetivo es darles un modelo mental compartido, una forma segura de arrancar y criterios concretos para saber si el Dia 1 termino bien.

La promesa de este sprint
Al finalizar la semana, una organizacion de demostracion podra registrar un paciente ficticio, agendarlo con un profesional, registrar un diagnostico, pedir y agendar rayos X, guardar un archivo de resultado, realizar un aviso manual y generar una factura claramente marcada como demostracion.

Como usar esta guia

1. Lean primero los capitulos 1 a 5 en conjunto. Son el vocabulario comun: si cada persona entiende algo distinto por contenedor, servicio o puerto, la integracion fallara aunque el codigo individual sea correcto.
2. El lunes sigan el capitulo 8 como agenda operativa. No comiencen por el capitulo de su nombre sin haber acordado primero contratos, nombres, puertos y variables.
3. Cada integrante usa luego su tutorial del capitulo 9. Las tareas hacen referencia a los identificadores del Excel: JJ-01, FR-04, AL-05, NI-05, JH-06, etc.
4. Mantengan abierto el Excel. El PDF explica como trabajar; el Excel sigue siendo la fuente de verdad de alcance, horas, estado, dependencias y evidencia.
5. Consulten los capitulos de operacion y troubleshooting antes de improvisar una solucion que rompa el aislamiento de servicios.
6. Antes de escribir una migracion V1, consulten MedicalSys_Diccionario_Datos_Microservicios_MVP.pdf. Ese documento es la fuente normativa de tablas, columnas, tipos, claves, referencias UUID y diagramas ER del modelo nuevo.

Reglas de seguridad de la capacitacion

- Usar unicamente pacientes, telefonos, documentos y estudios ficticios.
- No utilizar credenciales reales del SIN, WhatsApp, correo, MinIO externo, Vault ni ninguna cuenta institucional.
- No subir archivos clinicos reales, aunque esten anonimizados de manera informal.
- Toda factura debe mostrar la leyenda NO VALIDA TRIBUTARIAMENTE y el estado DEMO_ACEPTADA.
- El comando `docker compose down -v` elimina datos locales. Solo se ejecuta cuando el equipo confirma que desea reiniciar la demo.

Lo que no haremos esta semana
No implementaremos Kafka, outbox/inbox, replicas de referencia, auditoria, marketing, WhatsApp automatico, SIAT real, Vault productivo, antivirus ni un visor DICOM. Es deuda visible, no trabajo olvidado.

Contenido

Antes de comenzar 2

Como usar esta guia . . . 2

Reglas de seguridad de la capacitacion . . . 2

1. Objetivo del sprint y forma de trabajar 7

1.1 El objetivo expresado sin tecnologia . . . 7

1.2 Que significa Hecho . . . 7

1.3 Capacidad y disciplina . . . 7

2. Docker desde cero: el modelo mental minimo 9

2.1 El problema que resuelve Docker . . . 9

2.2 Imagen, contenedor, volumen y red . . . 9

2.3 Host versus contenedor . . . 9

2.4 El ciclo de vida que usaremos . . . 9

2.5 Como leer docker compose ps . . . 10

3. La arquitectura local de MedicalSys 11

3.1 Por que una base por servicio . . . 11

3.2 Una instancia MySQL, nueve bases logicas . . . 12

3.3 Fuente normativa del nuevo modelo de datos . . . 12

3.4 Puertos internos y bases acordadas . . . 13

3.5 Las tres redes . . . 13

4. Como se comunican los microservicios 15

4.1 Comunicacion sincrona para este MVP . . . 15

4.2 Recorrido de una solicitud . . . 15

4.3 Contrato: la frontera entre equipos . . . 15

4.4 Formato de error comun . . . 16

4.5 Codigos HTTP que usaremos . . . 16

4.6 Timeouts, reintentos e idempotencia . . . 17

5. Identidad, JWT y aislamiento por organizacion 18

5.1 Que contiene un JWT . . . 18

5.2 Responsabilidades del gateway . . . 18

5.3 Defensa dentro de cada servicio . . . 18

5.4 Rutas publicas e internas . . . 18

6. La plantilla comun de un microservicio 19

6.1 Estructura propuesta del repositorio . . . 19

6.2 Contenido minimo de cada servicio . . . 19

6.3 Configuracion por variables . . . 19

6.4 Dockerfile de referencia . . . 20

6.5 Compose de referencia . . . 21

6.6 Migraciones Flyway . . . 21

7. Acuerdos de equipo antes de escribir negocio 22

7.1 Reunion de contratos de las 09:30 . . . 22

7.2 Como compartir cambios sin bloquearse . . . 22

7.3 Daily de 15 minutos . . . 22

8. Dia 1: tutorial operativo paso a paso 23

8.1 Resultado obligatorio al cerrar el dia . . . 23

8.2 Agenda del dia . . . 23

8.3 Paso 1 - Verificar herramientas . . . 23

8.4 Paso 2 - Recorrer el Excel . . . 24

8.5 Paso 3 - Trabajar sin esperar la plantilla . . . 24

8.6 Paso 4 - Levantar infraestructura primero . . . 24

8.7 Paso 5 - Construir un servicio a la vez . . . 24

8.8 Paso 6 - Verificar aislamiento de datos . . . 25

8.9 Paso 7 - Cierre del dia . . . 25

9. Tutorial del Dia 1 por integrante 26

9.1 Juanjo - ms-catalogos y ms-criptografia . . . 26

9.2 Franco - ms-tenencia, ms-historial y ms-facturacion . . . 27

9.3 Alvaro - ms-identidad y ms-agenda . . . 28

9.4 Nicole - ms-pacientes y ms-documental . . . 29

9.5 Jhon - ms-profesionales, plantilla, Compose y gateway . . . 30

10. Operacion diaria con Docker 31

10.1 Rutina de inicio . . . 31

10.2 Rutina despues de cambiar codigo . . . 31

10.3 Rutina de diagnostico . . . 31

10.4 Volumenes y reinicio limpio . . . 31

10.5 Logs utiles . . . 31

11. Plan resumido de los Dias 2 a 5 32

11.1 Dia 2 - maestros e identidad . . . 32

11.2 Dia 3 - cita, encuentro y orden . . . 32

11.3 Dia 4 - documento, aviso y factura . . . 32

11.4 Dia 5 - reproducibilidad y Review . . . 33

12. Troubleshooting para principiantes 34

12.1 Arbol de decision rapido 34

12.2 Cuando pedir ayuda 34

13. Definition of Done y preparacion de la Review **35**

13.1 Checklist por servicio 35

13.2 Checklist del incremento 35

13.3 Guion de Review 35

Anexo A. Hoja rapida de comandos **36**

Anexo B. Variables y nombres recomendados **37**

Anexo C. Glosario minimo **38**

Referencias y fuentes **39**

Los numeros de pagina se actualizan automaticamente durante la generacion del documento.

1. Objetivo del sprint y forma de trabajar

Scrum no exige crear un producto MVP distinto cada semana. Exige que el sprint produzca un incremento usable, integrado y revisable. Para MedicalSys adoptaremos una regla mas fuerte: cada Sprint Review debe mostrar una rebanada vertical que atraviese varios servicios y tenga sentido para una persona usuaria.

1.1 El objetivo expresado sin tecnologia

El objetivo no es levantar diez proyectos Spring. El objetivo es demostrar el recorrido de una atencion medica. Los contenedores, las bases y el gateway son medios para lograrlo. Si el viernes hay diez health checks verdes pero no se puede completar el recorrido, el sprint no esta terminado.

Etapa	Actor	Resultado visible	Servicios principales
1. Preparar la clinica	Administrador	Organizacion, sucursal, prestaciones y profesional	tenencia, catalogos, profesionales, identidad
2. Recibir al paciente	Recepcionista	Paciente ficticio registrado	pacientes
3. Reservar	Recepcionista	Cita y consultorio sin solape	agenda
4. Atender	Profesional	Encuentro y diagnostico cifrado	historial, criptografia, catalogos
5. Pedir estudio	Profesional y recepcionista	Orden y cita de rayos X	historial, agenda
6. Guardar resultado	Recepcionista/profesional	Archivo y referencia en historial	documental, historial
7. Avisar	Recepcionista	Mensaje copiado y enviado fuera del sistema	proceso manual
8. Cobrar	Recepcionista	Factura local de demostracion	facturacion, tenencia, pacientes

1.2 Que significa Hecho

Una tarea no esta Hecha porque el controlador responde en la computadora del autor. Debe cumplir la Definition of Done del Excel y, como minimo, tener migracion repetible, health check, prueba clave, contrato actualizado, autorizacion basica, ausencia de datos reales y una llamada integrada desde su consumidor.

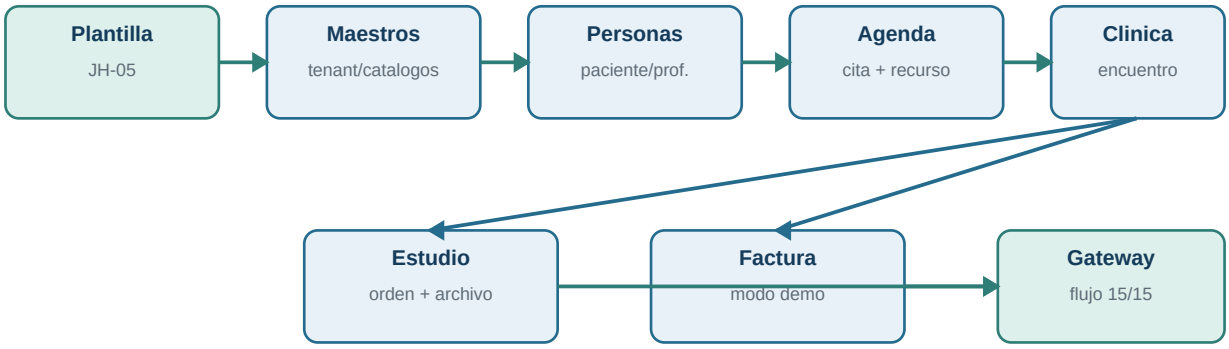
> **Frase que no aceptaremos**
> En mi microservicio funciona. La respuesta correcta es: funciona desde el gateway o desde el consumidor acordado, con una base limpia y con evidencia repetible.

1.3 Capacidad y disciplina

El Excel compromete 120 horas netas: 24 por integrante. Las otras horas de la semana quedan absorbidas por reuniones, aprendizaje, revisiones, esperas y correcciones. No llenen ese margen con funcionalidades nuevas. El margen es lo que hace posible terminar.

- Limite de trabajo en curso: una tarea principal por persona.
- Si alguien queda bloqueado mas de 30 minutos, actualiza el Excel y pide ayuda.
- Integracion dos veces al dia: antes del almuerzo y antes del cierre.
- Jueves al mediodia se congela el alcance. Desde ese momento solo se corrigen fallos del flujo.
- El viernes se demuestra desde datos limpios. No se prepara manualmente la base minutos antes de la Review.

Camino critico de la semana



Si una flecha se bloquea, no se inicia trabajo aguas abajo: se ayuda al bloqueo.

2. Docker desde cero: el modelo mental minimo

2.1 El problema que resuelve Docker

Sin Docker, cada integrante instala Java, MySQL, PostgreSQL, Redis y MinIO de manera distinta. Una version o puerto diferente puede producir errores que no aparecen en otra computadora. Docker empaqueta cada proceso con su sistema de archivos y configuracion, y Docker Compose describe como arrancan juntos.

Idea central
Docker no reemplaza el programa. Ejecuta el programa dentro de un proceso aislado y reproducible llamado contenedor.

2.2 Imagen, contenedor, volumen y red

Concepto	Explicacion sencilla	Ejemplo MedicalSys	Error habitual
Imagen	Plantilla inmutable con archivos y comando de arranque.	Imagen de ms-pacientes construida desde su Dockerfile.	Pensar que una imagen ya esta ejecutandose.
Contenedor	Instancia en ejecucion de una imagen.	Proceso ms-pacientes de este compose.	Editar archivos dentro y esperar conservarlos.
Volumen	Almacenamiento persistente gestionado por Docker.	Datos de MySQL o objetos de MinIO.	Eliminar volumen sin saber que contiene.
Red	Espacio privado donde contenedores se encuentran por nombre.	medicalsys-interna.	Usar IP fija o localhost para otro servicio.
Puerto de contenedor	Puerto que escucha el proceso dentro de su red.	ms-pacientes:8085.	Confundirlo con el puerto del host.
Puerto publicado	Puente entre la computadora y un contenedor.	localhost:8080 hacia el gateway.	Publicar todas las bases y APIs.
Compose	Archivo que declara servicios, redes, volumenes y variables.	compose.yaml en la raiz.	Usarlo como sustituto de una arquitectura productiva.

2.3 Host versus contenedor

La palabra host se refiere a la computadora donde corre Docker Desktop. Cada contenedor tiene su propio espacio de red. Por eso hay dos direcciones distintas para el mismo destino:

Quien llama	Direccion correcta	Motivo
Postman en la computadora	http://localhost:8080	El gateway publica el puerto 8080 al host.
Gateway dentro de Docker	http://ms-pacientes:8085	Docker resuelve el nombre del servicio en la red interna.
ms-agenda a pacientes	http://ms-pacientes:8085/internal/v1/...	La llamada no sale al host.
ms-pacientes a su BD	jdbc:mysql://mysql:3306/patients_db	mysql es el nombre del servicio de datos.

Error numero uno
Dentro de ms-agenda, http://localhost:8085 apunta al mismo contenedor de agenda. No apunta a ms-pacientes. Para hablar con pacientes se usa el nombre DNS del servicio: http://ms-pacientes:8085.

2.4 El ciclo de vida que usaremos

Comandos seguros de uso diario

```
# Validar la configuracion antes de arrancar
docker compose config

# Construir imagenes y levantar en segundo plano
docker compose up -d --build

# Ver estado y health
docker compose ps

# Seguir logs de un servicio
docker compose logs -f --tail=100 ms-pacientes

# Reconstruir solo el servicio modificado
docker compose up -d --build ms-pacientes

# Detener sin borrar datos
docker compose down
```

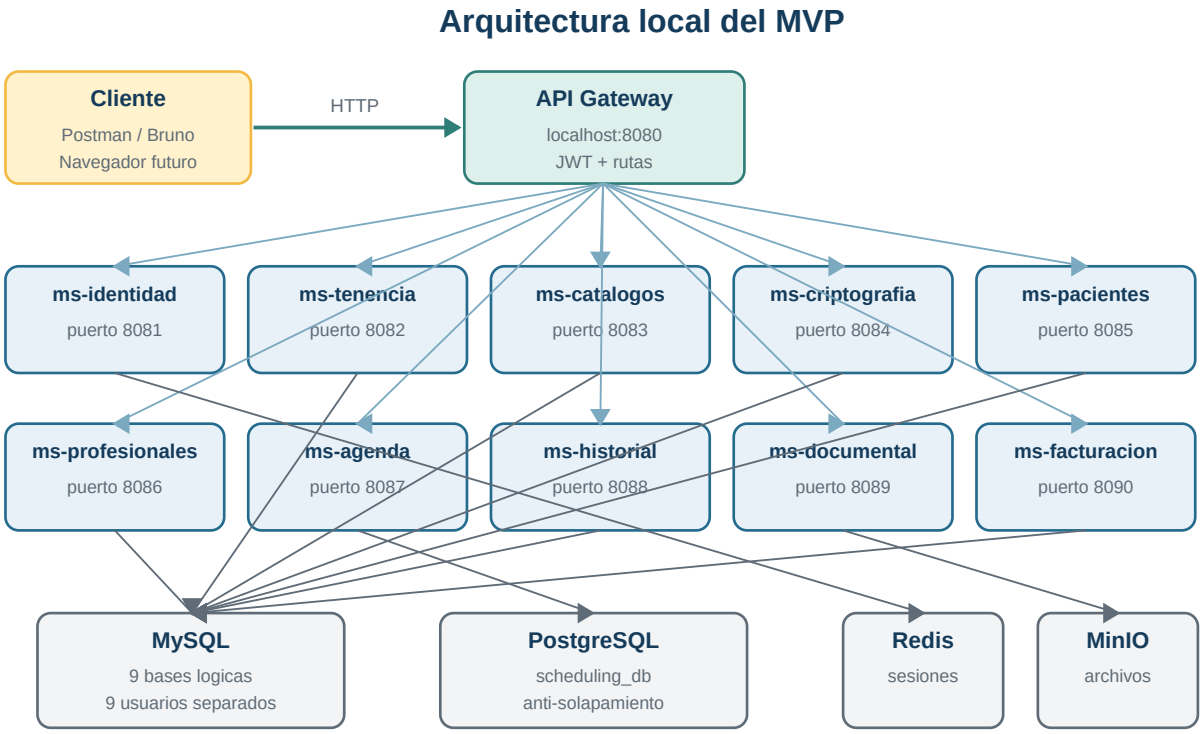
Comando destructivo local
docker compose down -v detiene el entorno y elimina los volumenes declarados. Sirve para probar un arranque limpio, pero borra las bases y archivos locales. Debe anunciarse en el canal del equipo antes de ejecutarlo.

2.5 Como leer docker compose ps

- running significa que el proceso sigue vivo; no necesariamente que la aplicacion esta lista.
- healthy significa que el healthcheck termino correctamente.
- unhealthy significa que el contenedor vive, pero la prueba de salud falla.
- exited significa que el proceso principal termino; el siguiente paso es leer sus logs.
- restarting indica un ciclo de fallos. No se arregla aumentando reinicios: se encuentra la causa en logs.

3. La arquitectura local de MedicalSys

El documento de arquitectura define trece microservicios y un gateway. El MVP usa diez: identidad, tenencia, catalogos, criptografia, pacientes, profesionales, agenda, historial, documental y facturacion. Notificaciones, marketing y auditoria quedan fuera. El gateway es una pieza adicional, no uno de los diez.



3.1 Por que una base por servicio

El limite mas importante no es el repositorio ni el puerto: es el dato. Cada servicio es el unico propietario que puede escribir su informacion. Otro servicio obtiene lo que necesita mediante un contrato HTTP. No consulta tablas ajenas y no crea claves foraneas hacia otra base.

Servicio	Es propietario de	No debe guardar
identidad	usuarios, roles, credenciales, sesiones	historia del paciente
tenencia	organizacion, sucursal, prestaciones y precios	citas o facturas
catalogos	CIE-10, especialidades, tipos de estudio	configuracion particular del tenant
criptografia	metadatos y uso de claves	encuentros clinicos
pacientes	identidad administrativa y contacto	credenciales o narrativa clinica
profesionales	matricula, especialidad y vinculo	agenda
agenda	horarios, citas y recursos	diagnosticos
historial	expediente, encuentro, diagnostico, orden y resultado	binarios de archivos
documental	objeto, hash, metadatos y vinculo	encuentro completo
facturacion	factura, detalle, correlativo y estado demo	narrativa clinica

3.2 Una instancia MySQL, nueve bases logicas

Para reducir el costo local, nueve servicios MySQL comparten un motor fisico. Esto no significa compartir una base. Cada servicio usa un nombre de base y un usuario distintos. Por ejemplo, patients_user recibe permisos solo sobre patients_db; billing_user solo sobre billing_db. Agenda conserva PostgreSQL porque necesita restricciones de rango para evitar solapes.

Regla de revision
Si las credenciales de ms-facturacion permiten SELECT sobre medical_records_db, la tarea no esta Hecha aunque el codigo nunca haga esa consulta.

3.3 Fuente normativa del nuevo modelo de datos

El diseno inicial de datos ya fue consolidado en **MedicalSys_Diccionario_Datos_Microservicios_MVP.pdf**. Ese documento describe exclusivamente las nuevas bases de los diez microservicios: 62 tablas relacionales, sus nombres tecnicos en ingles, equivalencias originales en espanol, columnas, tipos, claves, restricciones, referencias externas, diagramas ER, Redis para sesiones y MinIO para archivos.

Fuente	Uso durante el Sprint 2	Regla de prioridad
MedicalSys_Diccionario_Datos_Microservicios_MVP.pdf	Crear migraciones, entidades y pruebas de persistencia	Es la fuente normativa del modelo nuevo
MedicalSys_Microservicios.pdf	Entender limites, responsabilidades y evolucion arquitectonica	No reemplaza el diccionario fisico
Base de Datos/sql y MedicalSys_BaseDeDatos.pdf	Consultar trazabilidad o una regla heredada excepcional	No implementar nombres espanoles ni FK entre servicios

Convencion aprobada para la implementacion
Los identificadores de base, tabla y columna se escriben en ingles con snake_case. Los nombres publicos de los servicios, como ms-tenencia, se conservan porque forman parte de la arquitectura. Los nombres espanoles aparecen en el diccionario solo como equivalencia. Los datos ficticios se distinguen por el entorno y las semillas: se usa nit, no nit_demo; document_number, no document_demo.

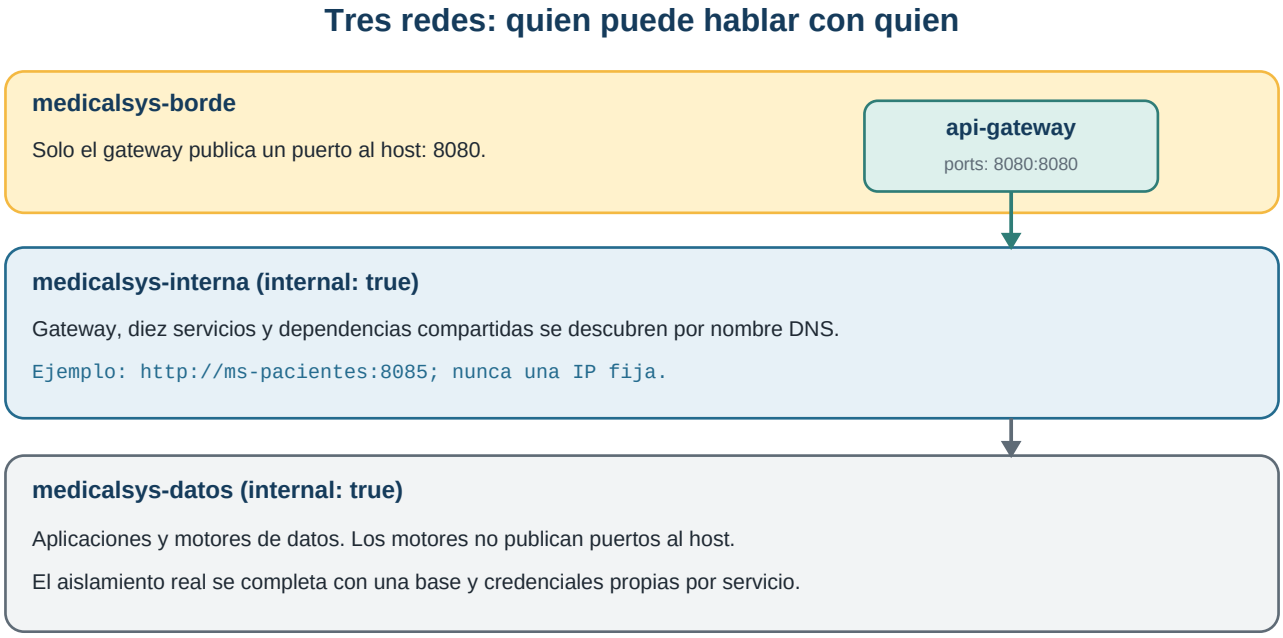
Servicio	Motor y base logica	Tablas objetivo en ingles
ms-identidad	MySQL: identity_db Redis: sesiones	user_account, role, permission, role_permission, user_role
ms-tenencia	MySQL: tenancy_db	subscription_plan, organization, subscription, branch, service_offering, plan_usage
ms-catalogos	MySQL: catalogs_db	specialty, icd10_catalog, medication, study_type, tax_catalog, medication_active_ingredient, catalog_sync
ms-criptografia	MySQL: cryptography_db	encryption_key, digital_certificate, key_usage
ms-pacientes	MySQL: patients_db	patient, notification_preference, patient_merge
ms-profesionales	MySQL: professionals_db	health_professional, professional_specialty, professional_organization
ms-agenda	PostgreSQL: scheduling_db	physical_resource, schedule_availability, schedule_block, appointment, resource_reservation, appointment_status_history
ms-historial	MySQL: medical_records_db	medical_record, allergy, medical_background, clinical_encounter y 8 tablas clinicas mas
ms-documental	MySQL: documents_db MinIO: binarios	document, document_link, consent_template, informed_consent, consent_event
ms-facturacion	MySQL: billing_db	billing_configuration, economic_activity, point_of_sale, invoice, invoice_item, payment y 6 tablas fiscales mas

Puerta de implementacion antes de Flyway
Cada responsable revisa en el diccionario el diagrama de su servicio y confirma nombres, tipos, PK, FK locales, referencias UUID externas, restricciones e indices. Si descubre una contradiccion, registra una decision y actualiza primero el diccionario; no improvisa un esquema distinto dentro de su rama.

3.4 Puertos internos y bases acordadas

Componente	Puerto	Base logica	Acceso host	Direccion interna
api-gateway	8080	-	Si: localhost:8080	http://api-gateway:8080
ms-identidad	8081	identity_db + Redis	No	http://ms-identidad:8081
ms-tenencia	8082	tenancy_db	No	http://ms-tenencia:8082
ms-catalogos	8083	catalogs_db	No	http://ms-catalogos:8083
ms-criptografia	8084	cryptography_db	No	http://ms-criptografia:8084
ms-pacientes	8085	patients_db	No	http://ms-pacientes:8085
ms-profesionales	8086	professionals_db	No	http://ms-profesionales:8086
ms-agenda	8087	scheduling_db (PG)	No	http://ms-agenda:8087
ms-historial	8088	medical_records_db	No	http://ms-historial:8088
ms-documental	8089	documents_db + MinIO	No	http://ms-documental:8089
ms-facturacion	8090	billing_db	No	http://ms-facturacion:8090

3.5 Las tres redes



Una red internal: true evita acceso directo desde fuera de Compose. En desarrollo, el gateway es la unica entrada publicada. Los servicios se conectan a la red interna y a la red de datos si necesitan un motor. Los motores solo se conectan a la red de datos.

No confundir red con permiso
Estar en la misma red solo permite intentar una conexion. El usuario de base, la contrasena y los GRANT determinan que datos puede leer. Se necesitan ambas capas.

4. Como se comunican los microservicios

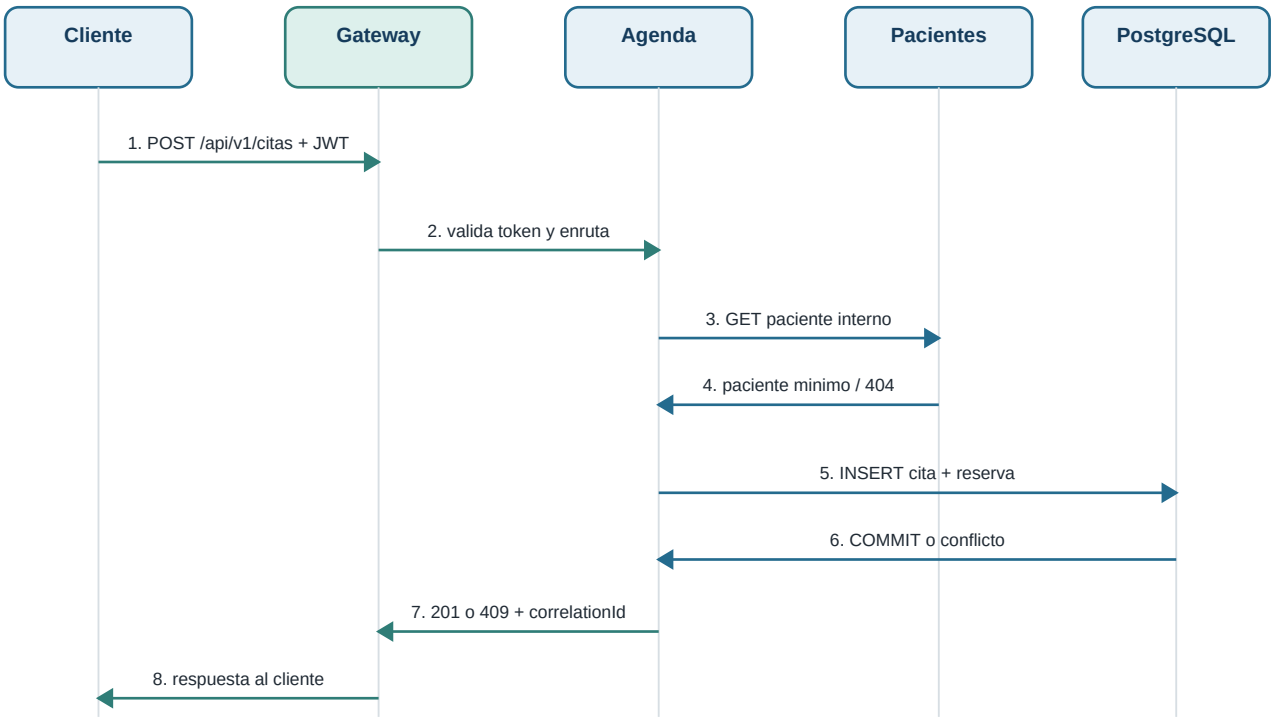
4.1 Comunicacion sincrona para este MVP

Durante esta semana usaremos HTTP REST sincrono. Sincrono significa que el servicio A llama al servicio B y espera una respuesta antes de terminar su propia operacion. Es apropiado cuando la respuesta es necesaria de inmediato, por ejemplo verificar que un paciente existe antes de crear una cita.

- Agenda valida patientId en ms-pacientes y professionalId en ms-profesionales.
- Historial consulta CIE-10 en ms-catalogos y cifra narrativa mediante ms-criptografia.
- Facturacion obtiene el DTO fiscal minimo del paciente y los precios de ms-tenencia.
- Documental devuelve documentId y hash; historial guarda esa referencia, no el binario.

> **Deuda consciente**
> La arquitectura destino usa eventos, outbox/inbox y replicas para reducir acoplamiento. No se implementan en este sprint porque el objetivo es aprender e integrar una rebanada vertical. Deben volver en el backlog posterior.

4.2 Recorrido de una solicitud



El correlationId viaja por todos los saltos y aparece en logs y errores. Permite reconstruir una solicitud sin revisar mensajes inconexos. Si el cliente no envia uno, el gateway lo genera.

4.3 Contrato: la frontera entre equipos

Un contrato define metodo, ruta, autenticacion, request, response y errores. El consumidor programa contra ese contrato, no contra la clase Java interna del productor. Cambiar el nombre de un campo sin avisar rompe a otro equipo aunque ambos servicios compilen.

Elemento	Acuerdo para el MVP	Ejemplo
Identificadores	UUID como texto	patientId: 7d1...
Fechas	ISO-8601 con zona; almacenar UTC	2026-08-24T14:00:00-04:00
Dinero	DECIMAL/BigDecimal, nunca float/double	precio: 150.00
Autenticacion	Authorization: Bearer JWT	No token en query string
Tenant	organizationId derivado del JWT	Gateway sobrescribe header interno
Trazabilidad	X-Correlation-Id	UUID por solicitud
Errores	JSON comun con code estable	APPOINTMENT_OVERLAP
Version	Rutas /api/v1 y /internal/v1	Cambios incompatibles crean v2

4.4 Formato de error comun

Ejemplo de error que el gateway puede devolver sin revelar detalles internos

```
{
  "timestamp": "2026-08-24T18:03:51Z",
  "status": 409,
  "code": "APPOINTMENT_OVERLAP",
  "message": "El profesional ya tiene una cita en esa franja",
  "correlationId": "2da15fa9-9f9c-4b62-b642-ef9bfb9fcf90",
  "path": "/api/v1/citas"
}
```

El campo code es mas estable que message. Las pruebas y clientes deben reaccionar a APPOINTMENT_OVERLAP, no a una frase que puede cambiar o traducirse.

4.5 Codigos HTTP que usaremos

Codigo	Uso	Ejemplo
200	Consulta o actualizacion exitosa	GET paciente
201	Recurso creado	POST cita
204	Operacion exitosa sin cuerpo	borrado logico si existiera
400	Request invalido	fecha final anterior a inicio
401	Token ausente, invalido o vencido	login requerido
403	Identidad valida sin permiso	paciente intenta crear profesional
404	Recurso no visible para el tenant	patientId inexistente o ajeno
409	Conflicto de estado o unicidad	CI duplicado o cita solapada
422	Regla de negocio no procesable	CIE-10 no permitido, si el equipo lo adopta
500	Fallo inesperado	Nunca incluir stack trace al cliente
503	Dependencia temporalmente no disponible	BD o servicio requerido caido

4.6 Timeouts, reintentos e idempotencia

- Toda llamada interna necesita timeout de conexion y lectura. Una espera infinita convierte una caida en bloqueo en cascada.
- No reintentar automaticamente POST no idempotentes sin una clave de idempotencia. Podria crear dos facturas o dos citas.
- Para el MVP, ante un 503 se devuelve un error claro y se conserva el correlationId. Los reintentos robustos quedan para el endurecimiento posterior.
- Una consulta GET puede reintentarse de forma acotada si el fallo es transitorio, pero no debe ocultar errores de contrato.

5. Identidad, JWT y aislamiento por organizacion

5.1 Que contiene un JWT

ms-identidad autentica credenciales y firma un JWT. Para el MVP, el token contiene como minimo userId, organizationId, roles, issuedAt y expiresAt. La firma permite verificar que nadie modifico esos claims.

Payload conceptual. Un JWT esta codificado, no cifrado: no colocar secretos dentro.

```
{
  "sub": "user-uuid",
  "organizationId": "org-demo-uuid",
  "roles": ["RECEPCIONISTA"],
  "iat": 1787590800,
  "exp": 1787594400,
  "iss": "medicalsys-ms-identidad"
}
```

5.2 Responsabilidades del gateway

1. Recibir la peticion externa por localhost:8080.
2. Permitir login y health segun la politica acordada; exigir Bearer token en el resto.
3. Verificar firma, emisor y expiracion del token.
4. Eliminar cualquier X-Organization-Id o X-User-Id enviado por el cliente y volver a crearlo desde claims confiables.
5. Generar o propagar X-Correlation-Id.
6. Aplicar la ruta correspondiente y reenviar al nombre DNS interno.
7. Devolver la respuesta preservando codigo HTTP y correlationId.

Fallo de seguridad comun
Nunca confiar en organizationId enviado libremente por Postman. Si una recepcionista puede cambiar el header y leer otra clinica, el aislamiento multi-tenant esta roto.

5.3 Defensa dentro de cada servicio

La arquitectura destino indica que cada servicio valida el token por si mismo. Para la primera integracion puede centralizarse la validacion criptografica en el gateway, pero cada servicio debe seguir aplicando autorizacion de rol y organizationId. El objetivo posterior es que tambien verifique la firma del JWT, de modo que una llamada que eluda el gateway no se vuelva confiable.

5.4 Rutas publicas e internas

Prefijo	Quien llama	Ejemplo	Regla
/api/v1	Cliente mediante gateway	/api/v1/pacientes	Roles y tenant obligatorios
/internal/v1	Otro microservicio	/internal/v1/pacientes/{id}	Solo red interna y autenticacion de servicio futura
/actuator/health	Docker y operadores	/actuator/health	Exponer solo salud basica

6. La plantilla comun de un microservicio

JH-05 es la tarea que desbloquea a todos. La plantilla evita que cada integrante invente una estructura, un error y una configuracion distintos. El lunes antes del mediodia debe estar disponible aunque solo contenga un endpoint de health y una migracion vacia.

6.1 Estructura propuesta del repositorio

Estructura recomendada. Si el repositorio ya tiene una convencion, conservarla.

```
medicalsys/
|-- compose.yaml
|-- .env.example
|-- README.md
|-- docs/
|   |-- contracts/
|   `-- demo/
|-- infra/
|   |-- mysql-init/
|   `-- minio-init/
|-- api-gateway/
`-- services/
    |-- ms-identidad/
    |-- ms-tenencia/
    |-- ms-catalogos/
    |-- ms-criptografia/
    |-- ms-pacientes/
    |-- ms-profesionales/
    |-- ms-agenda/
    |-- ms-historial/
    |-- ms-documental/
    `-- ms-facturacion/
```

6.2 Contenido minimo de cada servicio

- pom.xml o build.gradle con la misma version de Java y Spring para todos.
- Maven Wrapper o Gradle Wrapper versionado: nadie depende de una instalacion global distinta.
- Dockerfile comun y .dockerignore.
- application.yml sin secretos y application-local.yml solo si la politica del repo lo permite.
- Migraciones Flyway en src/main/resources/db/migration.
- Endpoint /actuator/health con la dependencia de base incluida en readiness.
- Manejador global de errores con el JSON comun.
- Filtro o interceptor de correlationId.
- Prueba de contexto, health y migracion.
- README corto: responsabilidad, puerto, base, variables, rutas y como probar.

6.3 Configuracion por variables

application.yml de referencia. Cada servicio cambia nombre y puerto.

```
server:
  port: ${SERVER_PORT:8085}

spring:
  application:
    name: ms-pacientes
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
  flyway:
    enabled: true

management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      probes:
        enabled: true
```

La configuracion no contiene localhost ni una contrasena real. Compose inyecta DB_URL, DB_USER y DB_PASSWORD. Un desarrollador que ejecuta Spring fuera de Docker puede usar un perfil local separado, pero no debe romper la configuracion del contenedor.

6.4 Dockerfile de referencia

Ejemplo multi-stage. Ajustar la imagen y comandos a la plantilla elegida.

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY .mvn .mvn
COPY mvnw pom.xml ./
RUN ./mvnw -q -DskipTests dependency:go-offline
COPY src src
RUN ./mvnw -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
RUN useradd --system --uid 10001 spring
COPY --from=build /workspace/target/*.jar app.jar
USER 10001
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Decision para aprender mas rapido
Durante el MVP todos pueden usar imagen JVM. La compilacion nativa de gateway y catalogos se retoma despues. Introducir dos pipelines de build el primer dia aumenta el riesgo sin mejorar la demo.

6.5 Compose de referencia

Fragmento educativo, no compose completo. El healthcheck debe usar una herramienta que exista en la imagen final.

```
name: medicalsys

services:
  api-gateway:
    build: ./api-gateway
    ports:
      - "8080:8080"
    networks: [medicalsys-borde, medicalsys-interna]

  ms-pacientes:
    build: ./services/ms-pacientes
    environment:
      SERVER_PORT: 8085
      DB_URL: jdbc:mysql://mysql:3306/patients_db
      DB_USER: patients_user
      DB_PASSWORD: ${PACIENTES_DB_PASSWORD}
    networks: [medicalsys-interna, medicalsys-datos]
    depends_on:
      mysql:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "-qO-",
             "http://localhost:8085/actuator/health"]
      interval: 10s
      timeout: 3s
      retries: 10

networks:
  medicalsys-borde: {}
  medicalsys-interna:
    internal: true
  medicalsys-datos:
    internal: true
```

depends_on no es suficiente
Compose puede arrancar en orden, pero un motor puede estar ejecutandose sin aceptar conexiones. Se usa healthcheck y condition: service_healthy para esperar readiness.

6.6 Migraciones Flyway

- V1__create_schema.sql crea tablas, indices y restricciones.
- V2__seed_demo.sql puede crear datos fijos si es idempotente y exclusivo del perfil demo.
- R__seed_catalogs_demo.sql es util para catalogos repetibles, pero debe evitar duplicados.
- Nunca editar una migracion que ya fue aplicada y compartida. Crear V3.
- La aplicacion debe fallar si la migracion falla. No continuar con un esquema incierto.

7. Acuerdos de equipo antes de escribir negocio

7.1 Reunion de contratos de las 09:30

Esta reunion dura 30 minutos y evita horas de correccion. Jhon facilita; cada productor y consumidor confirma su parte. Las decisiones se escriben en docs/contracts y en la hoja Contratos del Excel.

Decision	Valor recomendado	Quien confirma
Java	misma version LTS y mismo wrapper	Jhon + todos
Nombres Compose	ms-identidad, ms-pacientes, etc.	Jhon
Puertos	8080 a 8090 segun la tabla	Jhon
IDs	UUID serializado como string	todos
Tiempo	UTC en BD; ISO-8601 con offset en API	Alvaro + Franco
Dinero	DECIMAL(12,2) / BigDecimal	Franco
Headers	Authorization y X-Correlation-Id; tenant desde JWT	Alvaro + Jhon
Error	timestamp, status, code, message, correlationId, path	Jhon
Versionado	/api/v1 y /internal/v1	todos
Estado inicial	Por hacer; cambiar solo con evidencia	todos

7.2 Como compartir cambios sin bloquearse

Si el equipo no tiene una convencion existente, usar ramas cortas por servicio y Pull Requests pequenos. La rama no debe vivir toda la semana sin integrarse.

Adaptar la rama base a la estrategia real del repositorio.

```
# Ejemplos de nombres de rama
sprint2/ms-identidad-alvaro
sprint2/ms-historial-franco
sprint2/ms-documental-nicole

# Secuencia recomendada antes de pedir revision
git status
./mvnw test
docker compose config
docker compose up -d --build ms-pacientes
docker compose ps
```

- Commits pequenos con una intencion: migracion, endpoint, prueba o contrato.
- No mezclar cambios de tres servicios en un commit si no es una correccion de integracion.
- No versionar .env, target, volumenes, archivos de MinIO ni dumps de base.
- La persona propietaria revisa cambios sobre su servicio; el consumidor revisa el contrato.
- Integrar antes del almuerzo y al cierre para detectar incompatibilidades temprano.

7.3 Daily de 15 minutos

Pregunta	Respuesta util	Respuesta que no ayuda
Que termine	JJ-01: health y tablas de catalogos; evidencia en logs.	Avance bastante.
Que hare	JJ-02 y JJ-05; necesito el DB user antes de las 10.	Seguire programando.
Que me bloquea	JH-05 no define formato de error; afecta dos pruebas.	Docker no funciona.

8. Dia 1: tutorial operativo paso a paso

El lunes no se intenta completar el flujo clinico. El objetivo del Dia 1 es crear el terreno comun: herramientas verificadas, contratos congelados, plantilla disponible, motores sanos, bases separadas, servicios arrancando, migraciones aplicadas y responsabilidades claras.

8.1 Resultado obligatorio al cerrar el dia

- docker compose config termina sin errores.
- MySQL, PostgreSQL, Redis y MinIO estan healthy.
- Los diez servicios y el gateway arrancan al menos con /actuator/health.
- Cada servicio conoce su puerto, base, usuario y variables.
- Las migraciones del primer bloque se aplican desde volumen limpio.
- Ninguna base ni servicio, salvo el gateway, publica un puerto al host en la configuracion de equipo.
- Los contratos minimos, IDs, headers y formato de error quedan escritos.
- El Excel refleja estado real, no optimista.

8.2 Agenda del dia

Hora	Actividad	Resultado	Responsables
08:30-09:00	Verificacion individual de herramientas	captura/salida de versiones y Docker activo	todos
09:00-09:25	Sprint Kickoff	objetivo, exclusiones y Done entendidos	lider + todos
09:25-10:00	Congelar contratos y convenciones	docs/contracts y Excel actualizados	Jhon facilita
10:00-12:00	Plantilla/compose y SQL en paralelo	JH-05 listo; migraciones preparadas	todos
12:00-12:30	Primer smoke conjunto	infra healthy y un servicio plantilla	Jhon + todos
14:00-15:30	Crear/adaptar cada servicio	health, variables y conexion a su base	cada propietario
15:30-16:30	Aplicar migraciones y seeds	tablas verificadas sin acceso cruzado	cada propietario
16:30-17:10	Integracion y revision por pares	logs limpios y contratos coherentes	pares asignados
17:10-17:30	Cierre y update del Excel	bloqueos del Dia 2 visibles	todos

8.3 Paso 1 - Verificar herramientas

Cada persona ejecuta y comparte solo el resultado, nunca tokens ni variables secretas.

```
docker version
docker compose version
git --version
java -version

# Desde la raiz del repositorio, cuando exista el wrapper
./mvnw -version
```

- Si docker version no puede conectar con el daemon, abrir Docker Desktop y esperar a que termine de iniciar.
- Si docker compose no existe pero docker-compose si, no mezclar ambos comandos. Actualizar la herramienta o acordar una unica variante.
- Si Java difiere entre integrantes, detenerse y acordar una version antes de compilar servicios.
- Confirmar espacio en disco. Las imagenes y volumenes pueden consumir varios GB.

8.4 Paso 2 - Recorrer el Excel

Hoja	Para que se usa el lunes	Quien la mantiene
Resumen_MVP	Recordar objetivo, exclusiones e hitos	lider/PO
Product_Backlog	No ampliar ni reescribir historias sin decision	PO + equipo
Sprint_Tareas	Tomar tarea, estado, horas, dependencia y evidencia	cada responsable
Por_Integrante	Revisar carga de 24 h y apoyos	lider
Flujo_Demo	Entender el resultado del viernes	Jhon + equipo
Contratos	Congelar rutas, roles, request y response	productor + consumidor
DoD_Riesgos	No declarar Hecho sin evidencia	todos
Fuentes_Supuestos	Recordar simplificaciones del MVP	lider/arquitectura

8.5 Paso 3 - Trabajar sin esperar la plantilla

Mientras Jhon termina JH-05, nadie queda inactivo. Cada propietario puede preparar su migracion SQL, ejemplos de request/response, nombres de variables y README. No debe copiar una plantilla antigua incompatible ni crear otro formato de error.

Persona	Trabajo paralelo 10:00-12:00	No hacer aun
Juanjo	SQL de catalogos y metadatos de clave; seeds demo	API criptografica completa
Franco	SQL de tenencia e historial; restricciones e indices	factura fiscal real
Alvaro	roles/usuarios y DDL PostgreSQL de agenda	flujo de login final
Nicole	DDL de pacientes/documentos y convencion de archivo	subida completa a MinIO
Jhon	plantilla, compose, motores, health y contratos	gateway con todas las rutas

8.6 Paso 4 - Levantar infraestructura primero

Los nombres deben coincidir con compose.yaml; ajustar si el equipo adopta otros.

```
# Ver la configuracion efectiva y variables faltantes
docker compose config

# Levantar solo motores y storage primero
docker compose up -d mysql postgres-agenda redis minio

# Ver estados
docker compose ps

# Ver logs si algo no esta healthy
docker compose logs --tail=150 mysql
docker compose logs --tail=150 postgres-agenda
docker compose logs --tail=150 minio
```

No diagnosticar por intuicion
Primero docker compose ps; luego logs del contenedor que falla; despues variables y healthcheck. Cambiar puertos al azar suele ocultar la causa y romper contratos.

8.7 Paso 5 - Construir un servicio a la vez

No reconstruir las diez imagenes para probar un cambio en una sola.

```
# Construir el servicio de pacientes
docker compose build ms-pacientes

# Levantarlo con sus dependencias ya sanas
docker compose up -d ms-pacientes

# Seguir sus ultimos logs
docker compose logs -f --tail=100 ms-pacientes

# Verificar mediante la red publicada solo si pasa por gateway;
# durante el Dia 1 puede verificarse el health desde Compose/logs.
docker compose ps ms-pacientes
```

8.8 Paso 6 - Verificar aislamiento de datos

1. Confirmar que el servicio usa el DB_URL acordado y no localhost.
2. Revisar en logs que Flyway aplico V1 sin errores.
3. Conectarse con el usuario propio solo mediante una herramienta local autorizada o un contenedor de administracion temporal.
4. Intentar listar una base ajena con ese usuario. Debe ser rechazado.
5. No publicar el puerto de MySQL como solucion permanente. Si se abre temporalmente para depuracion, retirarlo antes de integrar.

8.9 Paso 7 - Cierre del dia

Checkpoint de las 17:10
Cada persona muestra: tarea del Excel, servicio en compose ps, log de migracion, health y una prueba de que usa su base. Si falta alguno, la tarea queda En curso y el bloqueo se registra.

- Actualizar estado y nota de bloqueo en Sprint_Tareas.
- Subir commits pequenos y abrir/revisar PR segun la estrategia acordada.
- Anotar variables nuevas en .env.example sin valores secretos.
- Dejar documentado el comando exacto para reproducir el servicio.
- Definir el primer objetivo del martes antes de retirarse.

9. Tutorial del Dia 1 por integrante

Cada subseccion indica que debe comprender la persona, en que orden trabajar, que evidencia entregar y que errores evitar. El orden respeta las tareas del Excel y el camino critico.

9.1 Juanjo - ms-catalogos y ms-criptografia

Objetivo del dia

Cerrar JJ-01, JJ-02 y dejar JJ-05 encaminada: catalogos arranca, crea tablas y carga datos demo; criptografia arranca con su esquema de metadatos y una clave de demostracion fuera del repositorio.

Lo que debes entender

- Catalogos contiene datos globales de referencia. Tenencia contiene las prestaciones y precios de cada organizacion. No mezclar ambos.
- Criptografia expone operaciones, no material de clave. Ninguna respuesta devuelve la clave maestra.
- El cifrado final se implementa el Dia 2. El Dia 1 se prepara el limite, configuracion y registro de uso.

Secuencia recomendada

1. Preparar el mapeo fuente -> ingles: especialidad -> specialty, catalogo_cie10 -> icd10_catalog, tipo_estudio -> study_type y catalogo_sin -> tax_catalog. Solo despues crear V1__create_catalogs.sql.
2. Preparar R__seed_catalogs_demo.sql idempotente: dos especialidades, tres CIE-10, X_RAY y dos codigos SIN simulados.
3. Crear ms-catalogos desde la plantilla, configurar puerto 8083 y catalogs_db/catalogs_user.
4. Levantar solo ms-catalogos, verificar Flyway y health. Reiniciar para confirmar que el seed no duplica.
5. Crear encryption_key, digital_certificate y key_usage exactamente como se definen en el diccionario nuevo. No agregar sufijo demo ni guardar la clave maestra en esas tablas.
6. Definir DEMO_MASTER_KEY como variable local en .env ignorado por Git y agregar solo el nombre a .env.example.
7. Crear ms-criptografia desde la plantilla, puerto 8084 y cryptography_db/cryptography_user.

Datos demo minimos

Catalogo	Ejemplos	Regla
especialidad	MEDICINA_GENERAL, RADIOLOGIA	codigo unico
CIE-10	tres codigos ficticios/educativos	guardar version de catalogo
study_type	X_RAY	codigo estable consumido por historial
catalogo SIN demo	CONSULTA, RAYOS_X	debe decir SIMULADO

Evidencia de cierre

- docker compose ps muestra ambos servicios y catalogos esta healthy.
- Logs de Flyway muestran migraciones aplicadas.
- Rearrancar catalogos no duplica seeds.
- git diff no contiene una clave real ni DEMO_MASTER_KEY con valor.
- README explica que Vault, rotacion y alta disponibilidad quedan fuera.

No hagas esto
No devuelvas la clave en un endpoint de depuracion y no la imprimas para comprobar que existe. Una clave filtrada en logs deja de ser secreta.

9.2 Franco - ms-tenencia, ms-historial y ms-facturacion

Objetivo del dia

Cerrar FR-01 y FR-02, preparar FR-04 y dejar el esqueleto fiscal listo: datos ficticios en tablas con nombres reales; esquema clinico minimo; facturacion sin acceso a datos clinicos.

Orden de prioridad

1. Tenencia primero: todos necesitan organizationId y servicios con precio/duracion.
2. Historial segundo: define expediente, encuentro, diagnostico, orden y resultado.
3. Facturacion tercero: solo esquema minimo; la generacion llega el Dia 4.

Tenencia paso a paso

- Crear subscription_plan, organization, subscription, branch, service_offering y plan_usage en tenancy_db siguiendo tipos, claves e indices del diccionario nuevo.
- Servicio debe incluir codigo, nombre, duracion_minutos, precio, activo y organization_id.
- Sembrar Clinica Demo, Central Branch, CONSULTATION y X_RAY como valores ficticios del entorno local.
- Precio usa DECIMAL y Java BigDecimal. Nunca double.
- El endpoint llega el Dia 2; el lunes basta migracion, seed, health y ejemplos de contrato.

Historial paso a paso

Tabla minima	Campos esenciales	Referencia externa
medical_record	id, patient_id, organization_id, created_at	patient_id sin FK cruzada
clinical_encounter	id, medical_record_id, appointment_id, professional_id, encrypted_narrative, signed	cita/profesional
diagnosis	id, encounter_id, icd10_code, primary_diagnosis	codigo de catalogos
study_order	id, encounter_id, patient_id, study_type_id, status	tipo de catalogos
study_result	id, order_id, document_id, status, summary	documental

No crear FK hacia pacientes, agenda, profesionales, catalogos o documental. Esos IDs son referencias de contrato. Si se necesita validar, se hace por HTTP en el momento correcto.

Facturacion paso a paso

- Crear billing_db y billing_user sin permisos sobre medical_records_db.
- Crear invoice, invoice_item y document_sequence segun el diccionario nuevo. La simulacion se expresa en datos/configuracion, no en los nombres.
- No implementar CUF/CUFD/CUIS reales ni copiar algoritmos fiscales sin validacion normativa.
- Planear la leyenda NO VALIDA TRIBUTARIAMENTE desde el modelo y response, no solo en la demo verbal.

Evidencia de cierre

- Clinica Demo y sus dos prestaciones existen tras un arranque limpio.
- medical_records_db contiene solo tablas del alcance minimo.
- billing_user no puede consultar medical_records_db.

- Todos los IDs externos estan documentados como UUID sin FK.

> **Riesgo personal de carga**
> Tienes tres servicios criticos. No intentes completar controladores de los tres el lunes. Cierra tenencia y los esquemas; pide apoyo temprano si JH-05 o los contratos se retrasan.

9.3 Alvaro - ms-identidad y ms-agenda

Objetivo del dia

Cerrar AL-01 y avanzar AL-05: usuarios/roles demo listos y esquema PostgreSQL capaz de impedir solapes. El login final y la consulta de disponibilidad llegan despues.

Identidad paso a paso

- Crear user_account, role y user_role en identity_db segun el diccionario nuevo; las sesiones viven en Redis y no en una tabla relacional.
- Sembrar ADMIN, RECEPCIONISTA y PROFESIONAL.
- Crear usuarios demo con hash de contrasena, nunca texto plano.
- Definir claims JWT con Jhon: sub/userId, organizationId, roles, iat, exp e iss.
- No usar organizationId enviado por el cliente como fuente de verdad.

Agenda paso a paso

Agenda usa PostgreSQL. El reto principal no es crear una tabla cita: es garantizar que dos transacciones concurrentes no reserven al mismo profesional o recurso en una franja superpuesta.

Idea de la restriccion de profesional. Ajustar nombres al modelo definitivo y probarla en PostgreSQL real.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE appointment
ADD CONSTRAINT no_professional_overlap
EXCLUDE USING gist (
  professional_id WITH =,
  tstzrange(starts_at, ends_at, '[)') WITH &&
)
WHERE (status IN ('CONFIRMED', 'COMPLETED'));
```

La notacion [) incluye el inicio y excluye el final. Asi una cita de 10:00 a 10:30 puede ser seguida por otra que comienza exactamente a las 10:30. Debe existir una restriccion analoga para resource_reservation.

Tablas del lunes

- schedule_availability: professional_id, weekday/date, starts_at, ends_at y organization_id.
- physical_resource: EXAM_ROOM_1 y X_RAY_1 con type y status.
- appointment: patient_id, professional_id, service_id, organization_id, starts_at, ends_at, status y order_id opcional.
- resource_reservation: appointment_id, resource_id, starts_at y ends_at.

Evidencia de cierre

- roles y usuarios demo se insertan sin contrasena en claro.
- scheduling_db aplica extension y constraints.
- Un SQL de prueba que inserta dos franjas superpuestas falla.
- Una cita que inicia cuando termina otra es aceptada.

No implementes tres capas para el mismo problema
La restriccion de PostgreSQL es la garantia final. La aplicacion puede validar antes para dar un mensaje amable, pero no sustituye la proteccion de la base frente a concurrencia.

9.4 Nicole - ms-pacientes y ms-documental

Objetivo del dia

Cerrar NI-01 y avanzar NI-05: paciente con CI unico y valores ficticios en columnas con nombres reales; esquema documental y bucket MinIO local bajo propiedad exclusiva de ms-documental.

Pacientes paso a paso

- Crear patient con id UUID, given_names, family_names, document_number, phone_e164, status y timestamps. Los valores son ficticios; los nombres no llevan sufijo demo.
- Aplicar unicidad al documento cuando corresponda; documentar como se representa SIN_DOCUMENTO si se necesita despues.
- No guardar contrasena: pertenece a identidad.
- No guardar diagnostico, alergia ni narrativa: pertenecen a historial.
- Preparar DTOs distintos: agenda necesita id/nombre; facturacion necesita id/nombre/documento; ningun consumidor recibe mas de lo necesario.

Documental paso a paso

Componente	Responsabilidad	Decision del MVP
document	name, mime_type, size_bytes, sha256_hash, object_key	sin contenido binario en MySQL
document_link	patient_id, order_id, owner_service	referencia blanda con UUID
MinIO	guardar el objeto	bucket medicalsys-demo
Validacion	tipo y tamano	PDF/JPG y DICOM pequeno; limite acordado
Antivirus	analisis de contenido	fuera del sprint

- Solo ms-documental recibe MINIO_ACCESS_KEY y MINIO_SECRET_KEY.
- object_key no usa el nombre original como unica clave. Usar UUID y conservar el nombre como metadata.
- Calcular SHA-256 al recibir el archivo y guardarlo junto a metadatos.
- El Dia 1 basta crear bucket, esquema, variables y health; la carga HTTP llega el Dia 3.

Evidencia de cierre

- CI demo duplicado queda impedido por restriccion de base.
- documents_db no contiene blob del archivo.
- Bucket existe y solo ms-documental tiene credenciales.
- README dice explicitamente sin antivirus y sin visor DICOM.

Privacidad desde el primer dia
No uses tu propio CI o telefono para que la demo se vea real. Usa valores claramente ficticios. Los datos de prueba suelen terminar en capturas, logs y repositorios.

9.5 Jhon - ms-profesionales, plantilla, Compose y gateway

Objetivo del dia

Tu trabajo desbloquea al equipo. Primero JH-05 y JH-06; despues JH-01. El gateway obtiene un esqueleto y health, pero las rutas completas se configuran el Dia 3.

Prioridad 1 - plantilla comun antes de las 12:00

1. Crear el servicio plantilla con version comun de Java/Spring y wrapper.
2. Agregar Actuator, Flyway, driver seleccionable, manejador de errores y correlationId.
3. Definir Dockerfile y .dockerignore.
4. Probar una imagen desde cero y el endpoint /actuator/health.
5. Documentar como copiar o parametrizar la plantilla sin duplicar identificadores incorrectos.
6. Entregar a todos y acompañar la primera adaptacion, no hacerla en secreto hasta la tarde.

Prioridad 2 - compose local

- Declarar tres redes y publicar solo 8080 del gateway.
- Declarar MySQL, PostgreSQL agenda, Redis y MinIO con volumenes y healthchecks.
- Crear nueve bases/usuarios MySQL mediante scripts de inicializacion.
- Declarar los diez servicios aunque algunos solo tengan health.
- Usar depends_on con condition: service_healthy donde corresponda.
- Agregar limites razonables y rotacion de logs si la version de Compose lo soporta en el entorno acordado.

Prioridad 3 - profesionales

- Crear health_professional, professional_specialty y professional_organization segun el diccionario nuevo.
- Matricula es unica; especialidad y organizationId son referencias externas.
- Preparar DTO minimo para agenda: id, organizationId, specialtyId y activo.

Gateway del lunes

Crear el proyecto, puerto 8080, health y una ruta de prueba. No esperar al viernes para descubrir que el gateway no construye. Las diez rutas y filtros JWT llegan despues de que identidad y contratos esten disponibles.

Evidencia de cierre

- Un integrante distinto puede crear un servicio desde la plantilla.
- docker compose up -d --build levanta infraestructura y servicios.
- docker compose ps no muestra ciclos de restart.
- Solo 8080 esta publicado en la configuracion compartida.
- Profesionales aplica su migracion y health.

Tu riesgo principal
No conviertas JH-05 en una plataforma perfecta. Entrega una plantilla pequena al mediodia y mejorala con el equipo. Una plantilla excelente entregada a las 17:00 bloquea a cuatro personas.

10. Operacion diaria con Docker

10.1 Rutina de inicio

No ejecutar comandos de Git que reescriban trabajo ajeno sin entender el estado local.

```
git pull --rebase   # solo si esta es la politica acordada
docker compose config
docker compose up -d
docker compose ps
```

10.2 Rutina despues de cambiar codigo

Reconstruir solo el servicio modificado.

```
./mvnw test
docker compose up -d --build ms-agenda
docker compose logs -f --tail=100 ms-agenda
docker compose ps ms-agenda
```

10.3 Rutina de diagnostico

1. Identificar el contenedor: docker compose ps.
2. Leer logs recientes: docker compose logs --tail=150 nombre.
3. Confirmar variables efectivas sin imprimir secretos completos.
4. Confirmar nombre DNS, red y puerto de contenedor.
5. Confirmar health de la dependencia.
6. Confirmar migracion y permisos de base.
7. Reducir el problema a una llamada o arranque minimo.
8. Registrar la causa y solucion en README o troubleshooting si puede repetirse.

10.4 Volumenes y reinicio limpio

Los datos sobreviven a docker compose down porque viven en volumenes. Esto es util durante desarrollo, pero puede ocultar que las migraciones no funcionan desde cero. El equipo debe hacer al menos un ensayo limpio antes del viernes.

Anunciar este reinicio y guardar solo datos ficticios reproducibles como seeds.

```
# ATENCION: elimina datos locales del proyecto Compose
docker compose down -v

# Reconstruir y levantar desde cero
docker compose up -d --build
docker compose ps
```

10.5 Logs utiles

- Incluir serviceName, correlationId y nivel.
- Registrar IDs de recursos, nunca narrativa clinica, contrasenas, JWT completos o claves.
- Un 4xx esperado no requiere stack trace de error del servidor.
- Un 5xx conserva la excepcion en logs internos y devuelve un mensaje generico al cliente.
- No resolver depuracion agregando System.out.println con objetos completos.

11. Plan resumido de los Dias 2 a 5

11.1 Dia 2 - maestros e identidad

Persona	Resultado	Tareas aproximadas
Juanjo	consultas de catalogos y cifrado AES-GCM	JJ-03 a JJ-07
Franco	endpoints de tenencia y esquema fiscal	FR-03, FR-05, FR-10
Alvaro	login JWT, usuarios, horarios y recursos	AL-02 a AL-06
Nicole	registro/busqueda de paciente	NI-02 a NI-04
Jhon	alta/listado de profesionales y smoke	JH-02 a JH-07

Checkpoint martes 17:00
Tenant, login, catalogos, paciente y profesional deben ser accesibles. Si no, no se abre mas alcance: se ayuda al servicio bloqueador.

11.2 Dia 3 - cita, encuentro y orden

- Agenda consulta huecos y crea cita validando paciente/profesional.
- Cita y reserva de recurso se guardan en una transaccion.
- La segunda reserva superpuesta devuelve 409.
- Historial crea/recupera expediente y registra encuentro con diagnostico.
- Narrativa se cifra antes de persistir.
- Historial crea orderId de RAYOS_X y agenda crea la segunda cita.

Checkpoint miercoles 17:00
Paciente -> cita -> encuentro -> orderId debe ejecutarse usando servicios reales, aunque el gateway aun tenga rutas parciales.

11.3 Dia 4 - documento, aviso y factura

- Documental sube archivo ficticio, valida, calcula SHA-256 y devuelve documentId.
- Historial registra resultado DISPONIBLE con documentId.
- Recepcionista obtiene telefono ficticio y texto para aviso manual.
- Facturacion consume paciente fiscal minimo y servicios/precios del tenant.
- Total y correlativo son automaticos; SIN es stub; salida tiene leyenda demo.
- Gateway valida JWT, propaga claims y enruta todas las rutas del guion.

Congelamiento jueves 12:00
Desde este punto no se agregan endpoints de conveniencia, UI, Kafka, filtros avanzados ni reportes. Solo se cierra el flujo y se corrigen fallos.

11.4 Dia 5 - reproducibilidad y Review

1. Reiniciar datos locales de manera coordinada.
2. Levantar todo siguiendo README.
3. Ejecutar la coleccion 15/15 sin editar IDs manualmente.
4. Verificar casos negativos: 401/403, CI duplicado, cita/recurso solapado, tipo de archivo invalido y factura demo.
5. Guardar evidencia tecnica permitida sin datos reales ni secretos.
6. Ensayar la narrativa de Review: problema, flujo, decisiones, limites y siguiente incremento.

12. Troubleshooting para principiantes

Sintoma	Causa probable	Que revisar	No hacer
Cannot connect to Docker daemon	Docker Desktop no inicio	abrir Docker Desktop; docker version	reinstalar todo inmediatamente
Connection refused a localhost	se uso localhost dentro del contenedor	usar nombre Compose y puerto interno	publicar todos los puertos
Unknown host ms-pacientes	servicios en redes distintas o nombre incorrecto	compose config y networks	usar IP fija
Access denied MySQL	usuario/password/GRANT incorrecto	variables, init SQL y base	usar root en la aplicacion
Flyway checksum mismatch	se edito migracion aplicada	crear migracion nueva o reinicio local coordinado	desactivar Flyway
Contenedor restarting	proceso Java termina	logs desde el primer error	subir restart indefinido
depends_on pero app falla	dependencia running no ready	healthcheck y service_healthy	agregar sleep fijo largo
401 desde gateway	token ausente, firma/emisor/exp incorrectos	Authorization Bearer y reloj	desactivar seguridad
403 con token valido	rol o tenant no permitido	claims y reglas de endpoint	cambiar rol a ADMIN para todo
Dos citas se crean	solo hay validacion en Java	constraint PostgreSQL y transaccion	sincronizar en memoria
Archivo desaparece al reiniciar	MinIO sin volumen	volume y object key	guardar archivo en container fs
Build muy lento	reconstruccion completa o dependencias	multi-stage, cache, servicio puntual	omitir tests siempre

12.1 Arbol de decision rapido

1. El contenedor no aparece: revisar compose config y nombre del servicio.
2. Aparece exited/restarting: leer logs; el problema esta en arranque, configuracion o migracion.
3. Aparece healthy pero la llamada falla: revisar ruta, gateway, JWT y nombre DNS del destino.
4. La llamada llega pero devuelve 4xx: el sistema esta funcionando; revisar contrato, rol o regla de negocio.
5. Devuelve 5xx: usar correlationId para localizar logs del salto exacto.
6. Solo falla despues de reiniciar: revisar volumenes, seeds, orden de readiness y dependencia de estado manual.

12.2 Cuando pedir ayuda

- Despues de 30 minutos sin una nueva hipotesis o evidencia.
- Cuando el cambio propuesto afecta compose, contratos compartidos, seguridad o una base ajena.
- Antes de ejecutar un comando destructivo o borrar un volumen de otra persona.
- Cuando una solucion exige saltarse el gateway, usar root o compartir una base.

> **Formato para pedir ayuda**
> Tarea: AL-05. Esperaba: segunda cita devuelve 409. Ocurre: ambas insertan. Evidencia: constraint no aparece en Flyway. Probe: logs y SQL adjuntos sin secretos. Hipotesis: migracion no se aplico. Esto permite ayudar mucho mas rapido que decir Docker falla.

13. Definition of Done y preparacion de la Review

13.1 Checklist por servicio

- [ ] Responsabilidad del servicio escrita en README.
- [ ] Puerto, DB, usuario y variables coinciden con Compose.
- [ ] Migraciones se aplican desde volumen limpio.
- [ ] Health devuelve UP y refleja dependencia de base cuando corresponde.
- [ ] Contrato request/response/error esta actualizado.
- [ ] Prueba de regla critica verde.
- [ ] Roles y organizationId se aplican.
- [ ] Logs tienen correlationId y no contienen secretos/datos clinicos.
- [ ] Otro integrante puede ejecutar el ejemplo.
- [ ] Tarea y evidencia se actualizaron en el Excel.

13.2 Checklist del incremento

- [ ] Solo el gateway esta expuesto al host.
- [ ] Diez servicios usan datos propios y no consultan bases ajenas.
- [ ] Coleccion crea sus IDs y recorre 15 pasos.
- [ ] Doble cita y doble recurso devuelven 409.
- [ ] Narrativa no aparece en claro en medical_records_db.
- [ ] Archivo vive en MinIO y historial guarda documentId.
- [ ] WhatsApp es un paso manual visible, no un envio fingido.
- [ ] Factura tiene total/correlativo y leyenda demo.
- [ ] No hay datos reales, secretos ni tokens en repositorio/evidencia.
- [ ] README permite levantar desde cero.

13.3 Guion de Review

Minuto	Quien	Contenido
0-2	lider/PO	Objetivo, alcance y limites del MVP
2-4	Jhon	Un comando, health y unica entrada 8080
4-7	Alvaro/Nicole	Login, paciente, profesional, disponibilidad y cita; mostrar 409
7-10	Franco/Juanjo	Encuentro, CIE-10, cifrado y orden RAYOS_X
10-12	Nicole/Franco	Archivo, resultado, aviso manual y factura demo
12-14	equipo	Limites, deuda visible y siguiente incremento
14-15	PO	Aceptar/rechazar segun criterios, no segun esfuerzo

Anexo A. Hoja rapida de comandos

Necesidad	Comando	Nota
Validar Compose	docker compose config	hacer antes de up
Levantar todo	docker compose up -d --build	construye cambios
Ver estado	docker compose ps	running no equivale a healthy
Logs recientes	docker compose logs --tail=100 servicio	no muestra secretos
Seguir logs	docker compose logs -f servicio	Ctrl+C no detiene contenedor
Reconstruir uno	docker compose up -d --build servicio	preferido durante desarrollo
Reiniciar uno	docker compose restart servicio	no reconstruye imagen
Detener	docker compose down	conserva volumenes
Borrar entorno	docker compose down -v	DESTRUCTIVO para datos locales
Listar volumenes	docker volume ls	no borrar por patron amplio
Ver configuracion	docker compose config --services	lista nombres efectivos
Ejecutar shell	docker compose exec servicio sh	solo si la imagen tiene shell
Tests Java	./mvnw test	antes de PR

Sobre imagenes minimas
Una imagen distroless puede no contener sh, curl o wget. Eso es una ventaja de seguridad. No agreguen un shell solo para depurar; usen logs, healthchecks compatibles o un contenedor temporal de diagnostico.

Anexo B. Variables y nombres recomendados

Servicio	DB	Usuario	Puerto	Variables adicionales
identidad	identity_db	identity_user	8081	REDIS_URL, JWT_ISSUER, JWT_KEY
tenencia	tenancy_db	tenancy_user	8082	-
catalogos	catalogs_db	catalogs_user	8083	-
criptografia	cryptography_db	cryptography_user	8084	DEMO_MASTER_KEY
pacientes	patients_db	patients_user	8085	-
profesionales	professionals_db	professionals_user	8086	CATALOGS_BASE_URL
agenda	scheduling_db	scheduling_user	8087	PATIENTS_BASE_URL, PROFESSIONALS_BASE_URL
historial	medical_records_db	medical_records_user	8088	CRYPTO_BASE_URL, CATALOGS_BASE_URL
documental	documents_db	documents_user	8089	MINIO_URL, MINIO_ACCESS_KEY, MINIO_SECRET_KEY
facturacion	billing_db	billing_user	8090	PATIENTS_BASE_URL, TENANCY_BASE_URL, SIN_MODE=DEMO

Los valores concretos de password viven en .env local o en un gestor de secretos. .env.example contiene nombres y valores no sensibles de ejemplo. JWT_KEY y DEMO_MASTER_KEY nunca se versionan con valores reales.

Anexo C. Glosario minimo

Termino	Definicion aplicada al proyecto
API	Contrato HTTP que permite a un componente usar capacidades de otro.
Endpoint	Metodo y ruta concretos, por ejemplo POST /api/v1/citas.
DTO	Objeto de transferencia; contiene solo campos del contrato, no una entidad JPA compartida.
Gateway	Puerta de entrada que valida, filtra y enruta peticiones.
JWT	Token firmado con identidad y claims; esta codificado, no cifrado.
Claim	Dato dentro del JWT, como organizationId o roles.
Tenant	Organizacion cliente cuyos datos deben quedar aislados.
Microservicio	Componente autonomo con responsabilidad y datos propios.
Contenedor	Proceso aislado creado desde una imagen.
Imagen	Plantilla inmutable usada para crear contenedores.
Compose	Descripcion declarativa de servicios, redes, variables y volumenes locales.
Healthcheck	Prueba repetida que informa si un contenedor o aplicacion esta saludable.
Readiness	Capacidad de aceptar trabajo; puede fallar mientras la app inicia o su BD no responde.
Volumen	Almacenamiento persistente fuera del ciclo de vida del contenedor.
DNS interno	Resolucion de nombres de servicio que ofrece Docker en una red Compose.
Migracion	Cambio versionado y repetible del esquema de base.
Idempotencia	Repetir una operacion produce el mismo efecto, no duplicados.
CorrelationId	Identificador comun para seguir una solicitud entre servicios.
Outbox	Patron posterior para publicar eventos sin perder cambios confirmados.
Walking skeleton	Recorrido minimo de extremo a extremo que prueba la arquitectura y el valor.

Referencias y fuentes

Esta guia adapta el diseño y backlog existentes a un tutorial de ejecucion. Las referencias oficiales se incluyen para que el equipo pueda verificar conceptos sin depender de memoria o fragmentos de internet.

MedicalSys_Diccionario_Datos_Microservicios_MVP.pdf
Fuente normativa del modelo nuevo: 62 tablas, nombres ingles/espanol, tipos, claves, referencias y diagramas ER.

MedicalSys_Microservicios.pdf
Secciones 5, 6, 9, 11 y anexos: limites, datos, redes, puertos y riesgos.

MedicalSys_BaseDeDatos.pdf
Modelo monolitico completo, convenciones, diagramas y diccionario de 73 tablas.

Base de Datos/sql/*.sql
DDL fuente con columnas, tipos, indices, restricciones, triggers, procedimientos y semillas que deben transformarse por servicio.

MedicalSys_Backlog_v2.pdf
Historias, dependencias, Definition of Done y plan previo.

MedicalSys_Product_Backlog_MVP_Sprint_1.xlsx
Fuente de verdad del sprint adaptado: 16 PBI, 59 tareas y 120 horas.

Docker Docs - What is a container?
https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Docker Docs - What is an image?
https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/

Docker Docs - Networking in Compose
https://docs.docker.com/compose/how-tos/networking/

Docker Docs - Startup order and service_healthy
https://docs.docker.com/compose/how-tos/startup-order/

Docker Docs - Persisting container data
https://docs.docker.com/get-started/docker-concepts/running-containers/persisting-container-data/

Spring Boot - Actuator endpoints
https://docs.spring.io/spring-boot/reference/actuator/endpoints.html

Spring Cloud Gateway - Reference
https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/index.html

Spring Security - Resource Server JWT
https://docs.spring.io/spring-security/reference/6.5/servlet/oauth2/resource-server/jwt.html

Cierre de la induccion
El exito del primer dia no se mide por lineas de codigo. Se mide por la cantidad de incertidumbre eliminada: mismos nombres, mismos contratos, mismas herramientas, datos aislados, arranque repetible y bloqueos visibles.